

```
STATUS

ACCOUNT

STATUS

class Banner
  attr_accessible :horiz, :link, :visible, :image, :position
  has_attached_file :image, styles: { vert: '220'

  before_create :assign_position

  def assign_position
    max = Banner.maximum(:position)
    self.position = max ? max + 1 : 0
  end
end
```

FULL STACK DEVELOPMENT

REACT – O PRINCÍPIO

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	7
O QUE VOCÊ VIU NESTA AULA?	9
REFERÊNCIAS	10

EMANSP

O QUE VEM POR AÍ?

Nesta aula, exploraremos como o React pode ser utilizado para criar layouts eficientes e responsivos para aplicações web. Você aprenderá sobre a estruturação de componentes de layout, uso de CSS e bibliotecas específicas para layout no React, como Styled Components e React Bootstrap.



HANDS ON

Será demonstrado como criar um layout básico utilizando React. Abordaremos desde a criação de um projeto React com Vite até a implementação de um layout responsivo com uso de flexbox e grid em CSS. Além disso, também exploraremos como integrar bibliotecas de componentes para agilizar o desenvolvimento.

Iniciando o projeto React com Vite

Começaremos criando um novo projeto React usando Vite. Vite é uma ferramenta moderna e rápida para projetos front-end que nos permite iniciar rapidamente. Abra o terminal e digite o seguinte comando:

```
npm create vite@latest my-react-layout -- --template react
```

Após criar o projeto, entre no diretório my-react-layout e instale as dependências com npm install.

Estrutura de diretórios

Organize os diretórios do projeto criando uma pasta components dentro de src. Dentro de components, crie três arquivos: Header.jsx, Footer.jsx e Main.jsx. Cada um desses arquivos representará um componente específico do layout.

Criando componentes de Layout

Vamos criar um componente funcional simples para cada parte do layout:

- Header.jsx: esse componente retornará um cabeçalho com um título.
- Main.jsx: aqui você colocará o conteúdo principal da página, como um texto de exemplo.
- Footer.jsx: este componente retornará um rodapé com informações como direitos autorais.

Utilizando CSS para estilização

Para cada componente de layout, crie um arquivo CSS correspondente, como Header.css, Main.css e Footer.css. Aqui, você definirá estilos básicos para cada parte do layout, como cores de fundo, alinhamento de texto e padding.

Montando o Layout na aplicação

No arquivo App.jsx, importe os componentes Header, Main e Footer. Organize-os na ordem em que devem aparecer na página. Utilize <div>s para criar uma estrutura de layout, aplicando estilos para posicionar corretamente cada componente.

Adicionando responsividade

Utilize técnicas como Flexbox ou CSS Grid no arquivo CSS principal (App.css) para tornar o layout responsivo. Por exemplo, você pode definir que o cabeçalho e o rodapé ocupem a largura total da tela, enquanto o conteúdo principal se ajusta de acordo com o tamanho da janela.

Executando e testando o Layout

Após implementar o layout, execute o projeto com `npm run dev` e abra o navegador para visualizar o resultado. Teste a responsividade redimensionando a janela do navegador e ajuste os estilos conforme necessário.

Estrutura do diretório

Para deixar tudo mais visível, temos uma estrutura parecida com a da figura 1.

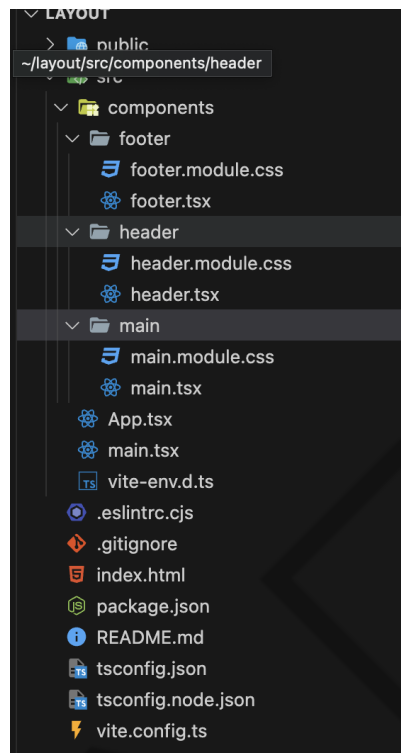


Figura 1 – Layout

Fonte: elaborada pela autora (2024)

SAIBA MAIS

Layouts são uma parte crucial do desenvolvimento front-end. Um bom layout proporciona uma experiência de usuário consistente e agradável. Discutiremos as melhores práticas para estruturar os componentes de layout, como cabeçalhos, rodapés e áreas de conteúdo, além de como garantir que eles sejam responsivos e adaptáveis a diferentes tamanhos de tela.

A importância da hierarquia visual: um layout eficaz usa a hierarquia visual para guiar o usuário através do conteúdo de forma natural. Isso pode ser feito usando diferentes tamanhos de fonte, cores ou até mesmo espaçamento. No React, você pode controlar esses aspectos facilmente através de CSS ou Styled Components para criar uma jornada visual atraente.

Componentes reutilizáveis - o coração do React: o React é famoso por sua capacidade de reutilizar componentes. Você pode criar um componente de cabeçalho ou rodapé uma única vez e usá-lo em várias páginas. Isso não só economiza tempo, mas também ajuda a manter a consistência do design em toda a sua aplicação.

Layout responsivo - uma necessidade, não uma opção: em um mundo com uma infinidade de dispositivos e tamanhos de tela, um layout responsivo não é mais um luxo, é uma necessidade. Utilize técnicas como media queries em CSS e flexbox para garantir que seu layout se adapte perfeitamente de um desktop para um tablet ou smartphone.

CSS Grid - o futuro dos Layouts: o CSS Grid é uma técnica relativamente nova que está revolucionando a maneira como criamos layouts complexos. Com o Grid, você pode criar designs que eram difíceis ou impossíveis com métodos antigos. É uma ferramenta poderosa em seu arsenal React para layouts mais dinâmicos e criativos.

A psicologia das cores no Layout: as cores desempenham um papel crucial no design. Elas podem evocar emoções, destacar elementos importantes e até influenciar decisões do usuário. No React, pensar cuidadosamente na paleta de cores do seu layout pode fazer uma grande diferença na eficácia geral do design.

Animações e transições para vida: animações sutis e transições podem adicionar um toque de elegância ao seu layout. No React, você pode usar bibliotecas como Framer Motion para adicionar animações sofisticadas que melhoram a interatividade sem comprometer o desempenho.

Acessibilidade - um Layout para todos: um bom layout não é apenas bonito, mas também acessível. No React, é importante considerar a acessibilidade, garantindo que seu site seja navegável para todos os usuários, incluindo aqueles com necessidades especiais.

Ambiente de desenvolvimento

- [Visual Studio Code](#);
- [Node](#), de preferência a versão LTS (obrigatório para evitar problemas com pacotes);
- [Typescript](#) (instalar em ambiente global);
- [Vite](#), uma ferramenta de desenvolvimento para a construção de aplicativos web modernos. No contexto do ecossistema ReactJS, o Vite desempenha um papel fundamental, oferecendo vantagens significativas durante o desenvolvimento e facilitando a criação de aplicações mais eficientes. Consulte no site alguns dos propósitos e benefícios do Vite no ecossistema ReactJS.

O QUE VOCÊ VIU NESTA AULA?

Focamos no desenvolvimento de layouts utilizando React, abordando desde a teoria até a prática. Discutimos diferentes estratégias e ferramentas que podem ser utilizadas para criar layouts eficientes e responsivos em aplicações React.

EMENDAS

REFERÊNCIAS

ADRIANO, T. **Typescript**. [s.l.]: Casa do Código, 2021.

BABEL. **Babel**. [s.d.]. Disponível em: <https://babeljs.io/>. Acesso em: 28 jun. 2024.

CARVALHO, T. **React: o que é e como funciona essa ferramenta?** Disponível em: <https://medium.com/reactbrasil/react-o-que-%C3%A9-e-como-funciona-essa-ferramenta-319922a8371c>. Acesso em: 28 jun. 2024.

REACT. **Documentação React**. [s.d.]. Disponível em: <https://react.dev/>. Acesso em: 28 jun. 2024.

PALAVRAS-CHAVE

Palavras-chave: Reactjs. Front-end. Biblioteca.

EMSE

POS TECH